

Security Notes & Secrets Handling

Satisfies the evaluation requirement: "Basic security considerations noted (network segmentation, RBAC, image scanning or policy notes)" and "Security notes and secrets handling explanation."

This document describes the security posture of the PoC, what is implemented, and — importantly — what is **deliberately out of scope** for a single-node proof-of-concept, stated honestly rather than hidden.

1. Secrets Handling (no secrets in the repo)

Principle: follow the 12-Factor App rule — *store config in the environment*, never in code or version control.

How it's implemented:

Layer	What	How
Application code	Reads all config from environment variables (<code>os.environ.get</code>)	No credentials hardcoded in <code>app.py</code>
Non-secret config	<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USER</code>	Kubernetes ConfigMap (<code>configmap.yaml</code>)
Secret config	<code>DB_PASSWORD</code>	Kubernetes Secret (<code>secret.yaml</code>), base64-encoded, injected as an env var
CI/CD credentials	VM host, SSH user, SSH private key	GitHub Secrets (<code>VM_HOST</code> , <code>VM_USER</code> , <code>VM_SSH_KEY</code>) — never printed in logs
Registry auth	ghcr.io push	GitHub's built-in <code>GITHUB_TOKEN</code> — scoped, ephemeral, auto-rotated

Verification: no plaintext passwords, tokens, or keys exist anywhere in the Git repository. The `Helm values.yaml` carries only a placeholder DB password (`changeme-in-ci`) which is overridden at deploy time.

Production hardening (out of scope here): use a dedicated secrets manager (e.g. Vault, AWS Secrets Manager, Sealed Secrets / External Secrets Operator) and enable encryption-at-rest for Kubernetes Secrets (`EncryptionConfiguration`).

2. RBAC (least privilege)

Application ServiceAccount (`serviceaccount.yaml`):

- The app runs under a **dedicated ServiceAccount** (`notes-api-sa`), not the namespace `default` account.
- `automountServiceAccountToken: false` — the Kubernetes API token is **not** mounted into the pod. The app never calls the Kubernetes API, so it is granted **zero** cluster permissions. This is least privilege in its purest form: if the pod were compromised, the attacker gains no Kubernetes API access from it.

- No Role/RoleBinding is attached — deliberately, because the app needs no API access. Granting an empty Role would be pointless; granting none is correct.

CI/CD least privilege:

- The deploy job connects over SSH as the **non-root ubuntu user** using a **scoped, user-owned kubeconfig** (`/home/ubuntu/.kube/config`, permissions 600).
- The pipeline is **not** granted root/sudo on the VM. If CI credentials leaked, the blast radius excludes host root. This was a deliberate choice over the simpler (but broader) approach of running deploy commands with `sudo`.

3. Network Segmentation (NetworkPolicy) — with an honest caveat

Implemented: a **NetworkPolicy** (`networkpolicy.yaml`) declaring that the PostgreSQL pod accepts ingress **only** from the Notes API pods, on port 5432. Intent: the database is not reachable from any other pod in the cluster.

Honest limitation (important): the default k3s CNI is **Flannel**, which does **not enforce NetworkPolicies**. Kubernetes accepts the policy object without error, but no traffic is actually blocked. The policy therefore expresses the *correct intent* and documents the desired segmentation, but is **not enforced** in this PoC.

To make it enforce (production): install a policy-capable CNI such as Calico (e.g. `k3s ... --flannel-backend=none` + Calico), after which the same policy would be enforced with no changes to the manifest.

This distinction between declaring and enforcing a NetworkPolicy is called out explicitly because silently shipping a non-enforcing policy would be misleading.

4. Container Image Security

Non-root container (`Dockerfile`):

- The image runs as a dedicated non-root user (`appuser`), not root. If the container is compromised, the attacker does not have root inside it — reducing blast radius.
- Built on `python:3.12-slim` — a minimal base image, reducing the attack surface and the number of packages that could carry vulnerabilities.

Image scanning in CI (`.github/workflows/build-deploy.yml`):

- Every image built by the pipeline is scanned with **Trivy** for CRITICAL and HIGH severity vulnerabilities before deploy. Currently report-only (non-blocking) as is appropriate for a PoC; in production the build would **fail** on CRITICAL findings (`exit-code: 1`).

Immutable image tags:

- Images are tagged with the git commit SHA (not just `:latest`), giving reproducible, auditable deploys and avoiding stale-image ambiguity.

5. Observed: Real-World Exploit Scanning

Within hours of exposing the app on a public Elastic IP, the `/metrics` endpoint recorded a stream of unsolicited 404 requests probing paths like `/vendor/phpunit/.../eval-stdin.php` and `/index.php` — automated bots scanning for known vulnerabilities. All were safely rejected (the app is Python/Flask, not PHP; those routes don't exist). This real observation reinforces why the hardening above matters: **public exposure attracts automated attacks within minutes**, and a minimal attack surface + non-root containers limit what any successful exploit could achieve. (Detailed in [docs/design-notes.md](#).)

Production follow-up: restrict the security group to known reviewer IPs, add a WAF / rate-limiting, and consider fail2ban-style IP blocking.

6. Summary Table

Control	Status	Notes
No secrets in repo	✓ Implemented	ConfigMap/Secret + GitHub Secrets
Secrets injected at runtime	✓ Implemented	12-factor; env vars
Non-root container	✓ Implemented	<code>appuser</code>
Minimal base image	✓ Implemented	<code>python:3.12-slim</code>
RBAC least privilege	✓ Implemented	dedicated SA, API token disabled
CI/CD non-root deploy	✓ Implemented	scoped kubeconfig, no host root
Image vulnerability scanning	✓ Implemented	Trivy in CI (report-only)
Immutable image tags	✓ Implemented	git SHA tags
Network segmentation	⚠ Declared, not enforced	Flannel doesn't enforce; needs Calico
Secrets encryption-at-rest	✗ Out of scope	Production: KMS/Vault
DB high availability	✗ Out of scope	Single-node PoC; DB is a SPOF

Philosophy: implement sensible defense-in-depth appropriate to a single-node PoC, and **document honestly** what is not covered rather than overstating the security posture.